Core entry points (Claude — and the same idea in any AI)

Source course: Associate · Module 1 · Claude Platform & Model Foundations · Core Entry Points
Goal: choose the right *container* for work before you type, so context does not tax you every session.

Prereqs: you have used a chat UI at least once. No API knowledge required.

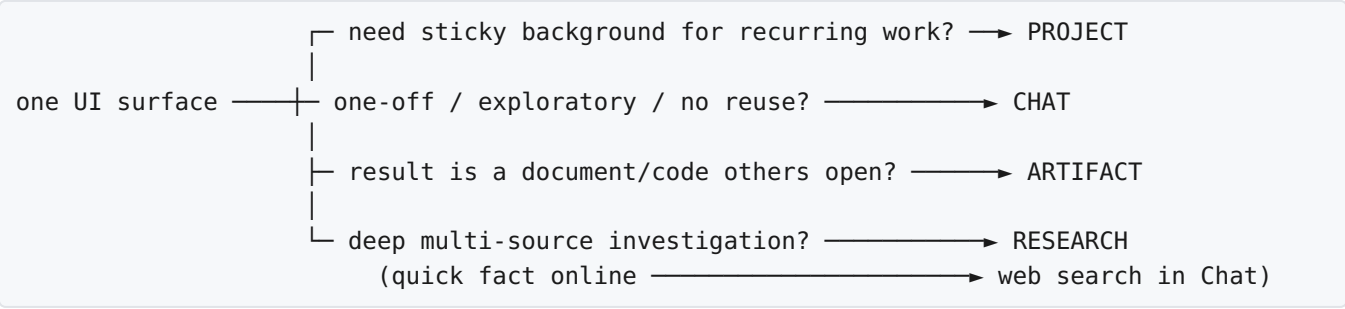
One-sentence definition

An **entry point** is *where* and *how* you start a task with Claude (or any AI product): it decides what context is sticky, what is disposable, and what form the output takes.

Why this exists

Models do not remember your job by magic. Persistence is a **product feature** (Projects, knowledge files, memory) or a **habit** (re-paste). Wrong container = you re-explain the same background every week. That is the “setup tax.”

High-level map



Fundamental unit: one *task with a context budget*.
Mechanism: your choice loads different context + tools + output surface.

Task type	Entry point
One-off question or quick task, no plan to reuse	**Chat**
Recurring work with stable context	**Project**

Task type	Entry point
Output is a deliverable someone will open and read	**Artifact**
Deep multi-source investigation or synthesis	**Research**
Quick current-information lookup	**Web search in Chat** (not Research)

These can **combine**: e.g. open a **Project**, run **Research**, ask for an **Artifact**.

The four entry points

1. Chat

What: default unstructured conversation. Saved in history; you can continue later.

What Chat gives you: continuity inside that thread; on many plans, **Memory** and past-chat search can carry *some* facts into new sessions.

What Chat does not give you: a Project's deliberate package — standing instructions + curated knowledge base shared by design across new threads in that workspace.

Use for: one-off questions, quick drafts, exploratory prompting, work that starts and ends in the session.

Outgrown Chat when: you open a new chat by pasting the same background paragraph you pasted last week.

Failure mode: "I'll just re-paste the brief" becomes a permanent process. Fix: promote to Project.

---

2. Projects

What: persistent workspace. Official Anthropic definition: self-contained workspace with its own chat histories and knowledge base.

A Project holds three things:

Piece	Job
Standing instructions	How Claude should behave in *every* chat in this Project (role, format, constraints)
Knowledge base	Docs, policies, notes, code — uploaded once; used as context without re-upload each session
Conversation list	Chats live under the Project, separate from global history

Critical mechanism (official): conversations inside a Project share instructions + knowledge base. They **do not** automatically share context with each other. If chat A learned a fact you need in chat B, put that fact in **project knowledge** (or instructions), not only in chat A.

Who can use Projects: all Claude accounts (including free). Free accounts: **max 5 projects**. Paid plans get larger knowledge capacity; when knowledge nears the context limit, Claude can enable **RAG** for that Project (paid).

Create (official steps):

1. Go to claude.ai/projects → **+ New Project**
2. Name + description (*Claude does not use the description as instructions*)
3. **Set project instructions** → save
4. **+** → upload knowledge files
5. Start chats *inside* the Project

Worth building if ≥2 of these are true:

1. Does the task **recur**?
2. Is the **background** stable across sessions?
3. Is the **output format** consistent?

Setup cost is usually recovered in **2–3 sessions**.

Failure modes:

Symptom	Likely cause	Fix
Claude “forgot” last week’s decision	Fact only lived in an old chat	Add to knowledge or instructions
Every Project feels the same	Vague instructions (“be helpful”)	Role + format + quality bar + don’ts
Wrong tone or secrets leak across work	One mega-Project for everything	Split by workstream / sensitivity
Knowledge ignored / noisy answers	Dump of huge unrelated PDFs	Curate; prefer short source-of-truth notes

---

3. Artifacts

What: deliverable surface. Claude puts the result in a **separate editable block** beside chat, not only as threaded prose.

Use for: draft documents, data tables, formatted reports, code, anything a *recipient* will open.

Use inline chat instead when: the answer is for *you* to act on inside the conversation (yes/no, next command, short explanation).

Failure mode: long report buried in chat history → hard to export/edit. Prefer Artifact for shippable work.

---

4. Research (vs web search)

Web search (Chat toggle): available on Claude plans as a per-chat control (Team/Enterprise may need an Owner to enable workspace-wide first). Good for **quick** current facts. Default on/off is not always specified in docs — check your UI.

Research: deeper mode (plan-gated; paid tiers). Multi-step search across many sources (and connected apps where enabled), then **synthesis** into a structured report with citations. Use when the job is investigation, not a one-shot lookup.

Rule of thumb:

- “What is X’s current pricing?” → Chat + web search
- “Compare X/Y/Z for our use case with sources” → Research
- Then put the report in an **Artifact** if someone else will read it

Failure mode: burning Research quota on a single fact that web search answers in one hop.

Worked examples (entry-point decisions)

Example A — Weekly status report (course worked comparison)

Wrong (Chat every Monday)
Coordinator pastes: project name, team structure, stakeholders, report format, last week’s open items. Claude writes a good report. Session ~40 min; ~12 min is re-loading context.

Right (Project once)

Put in Project	Put in Monday’s chat only
Team structure, stakeholders	This week’s updates
Report format template	Numbers that changed this week
Escalation rules in **instructions**	One-off exceptions

Session drops toward ~25 min. Quality same; **setup tax** gone.

Project instructions sketch:

```
You are the project coordinator’s status-report co-author.
Always output sections: Summary | Progress | Risks | Asks | Next week.
Escalation: flag any risk that blocks external launch as BLOCKER in bold.
```

```
Never invent metrics; leave gaps as TODO if I did not provide data.
Tone: neutral, executive-ready, no hype.
```

Example B — Learning a course module (your Associate track)

Project name: `Claude Associate M1 Foundations`

Knowledge base: syllabus screenshots or text, your notes, glossary, this vault's related files.

Instructions sketch:

```
You are a patient tutor for Claude Platform foundations.
Teach from first principles: definition → why → mechanism → when to use.
After each concept, ask me 2 check questions before continuing.
Prefer short tables over long prose.
If I am wrong, correct me and give one counter-example.
Do not invent product features; say "verify in UI/docs" when unsure.
```

Session pattern:

1. New chat per module section (Entry Points, Capability Layer, …)
2. After a good explanation: "summarize what I must memorize in 8 bullets" → **Artifact**
3. End of week: one chat "quiz me on Module 1" using only Project knowledge

Why Project: recurring study, stable syllabus, consistent "tutor" format.

Example C — Morning goals / brain dump (real operator pattern)

From Sarah Tavel · Nat Emodi "Morning Goals": a Project with structured standing instructions turns a voice or rambling memo into to-dos, meeting questions, and draft messages; strategy docs sit in knowledge.

Mechanism: stable *structure* in instructions + stable *company context* in knowledge + volatile *today's dump* in the chat.

Example D — Client or engagement knowledge base

Project per client or engagement (not one global Project).

Knowledge	Instructions
Scope email (redacted), stack notes, glossary of internal terms	Role (e.g. engagement scribe), report template, out-of-scope rules, "never invent findings"

Chat uses: daily recon notes, “rewrite this finding for the client.”
Artifact uses: weekly update PDF-style markdown.
Research uses: multi-source CVE / vendor comparison before a module (paid).

Example E — Bug bounty / lab notes workstream

Project: `Notes vault — AI & web foundations`
Knowledge: your `AGENTS.md` tone rules, checklists, past module notes.
Instructions: first-principles operator style; frugal; no AI slop; authorized-testing disclaimer when offensive.

New chats: “expand Core Entry Points with examples” vs “quiz me.”
Do **not** mix unrelated personal life chats into this Project (keeps memory/knowledge clean on plans that use project-scoped memory).

Example F — Code + docs deliverable

1. **Project** with repo subset or design doc in knowledge
2. Chat: “implement feature X following the design”
3. **Artifact:** final README or patch explanation for the PR

If the task is one throwaway script with no reuse → plain **Chat** is enough.

Example G — When Research is the right first move

“Map how Anthropic documents Projects, RAG, and memory, cite official help articles, and list open questions.” → **Research**, then save synthesis into Project knowledge so next week’s chats reuse it.

Fine-tune a Project for learning and productivity

Treat a Project like a **small product** you maintain.

1. Write instructions like a system prompt (not a vibe)

Minimum skeleton:

```
ROLE: who Claude is in this workspace
AUDIENCE: who the output is for
OUTPUT SHAPE: headings / tables / length limits
QUALITY BAR: what “done” means
HARD CONSTRAINTS: never invent X; cite sources; authorized scope only
PROCESS: e.g. ask clarifying questions first; quiz me after concepts
```

Weak: “Be helpful and professional.”
Strong: concrete sections, failure rules, and examples of good/bad output.

2. Curate knowledge (quality > bulk)

Upload: stable source-of-truth (SOPs, syllabus, brand voice, schema, past good reports).
Do not upload: secrets you should not store in a third-party SaaS, huge dumps of junk, every chat export.

Prefer one short `SOURCE_OF_TRUTH.md` you update over 40 stale PDFs.

Paid plans: large libraries can switch to **RAG** — still curate; garbage-in remains garbage-out.

3. One workstream per Project

Split when:

- different stakeholders or confidentiality
- different output formats (legal memo vs marketing copy)
- different “roles” you want Claude to play

Free tier: only **5** Projects — choose the five highest-ROI workstreams.

4. Promote discoveries into sticky memory

When a chat produces a rule you will need forever:

1. Edit **project instructions**, or
2. Add/update a knowledge file

Official note: moving chats in/out of Projects also affects **project-scoped memory** on plans that support memory.

5. Tune in a tight loop

```
use Project for real task
  → note one failure (wrong format, missing fact, too verbose)
  → change ONE thing (instruction line or one knowledge file)
  → new chat, same task, compare
```

Do not rewrite everything at once.

6. Session hygiene

- New chat when the topic shifts (reduces context pollution)
- Keep volatile data in the message; keep stable data in knowledge
- Star the Project for left-nav access
- Archive completed workstreams instead of deleting history you may need

7. Combine entry points deliberately

Stage	Entry
Ground once	Project setup
Investigate	Research (or web search)
Draft shippable	Artifact
Quick side question	separate Chat (or Project chat if it needs the same KB)

8. Map to other AI products (same principles)

Claude	Similar idea elsewhere
Chat	ChatGPT / Gemini thread
Project + instructions + knowledge	ChatGPT Projects; custom GPTs; Grok `AGENTS.md` + rules + skills
Artifact	Canvas / docs side panels / code preview
Research	“Deep research” modes in other vendors

The **names** change; the decision (ephemeral vs sticky vs deliverable vs investigate) does not.

Decision checklist (30 seconds)

Before you type:

1. Will I do a version of this again with the same background? → **Project**
2. Is the answer only useful inside this thread? → **Chat**
3. Will a human open a file-like result? → ask for **Artifact**
4. Do I need multi-source synthesis, not one fact? → **Research**
5. Am I about to paste last week’s paragraph again? → stop → **Project**

Official Claude references

Topic	URL
What are Projects?	https://support.claude.com/en/articles/9517075-what-are-projects
Create & manage Projects	https://support.claude.com/en/articles/9519177-how-can-i-create-and-manage-projects
Projects announcement	https://www.anthropic.com/news/projects
RAG for Projects (paid)	https://support.claude.com/en/articles/11473015-retrieval-augmented-generation-rag-for-projects

Topic	URL
Chat search & memory	https://support.claude.com/en/articles/11817273-use-claude-s-chat-search-and-memory-to-build-on-previous-context
Web search vs extended thinking vs Research	https://support.claude.com/en/articles/11095361-when-should-i-use-web-search-extended-thinking-and-research
Projects home	https://claude.ai/projects

Blogs & practical writeups

Piece	Why read it
Getting started with Claude Projects (Sarah Tavel / Nat Emodi)	Full **Morning Goals** instruction template + real daily workflow
Claude Projects complete guide (Melissa Onwuka, Medium)	Setup + why detailed custom instructions beat “be helpful”
Claude Projects tutorial — working knowledge base (Enterprise DNA)	Production-minded instructions and document ingestion
Northeastern — class Q&A Project	Learning-focused knowledge + instruction setup
Extended thinking vs web search vs Research (Waboom)	Clear mode comparison with pointer to official support article
Addy Osmani — AI coding workflow 2026	How Projects/context fit a durable engineering workflow (adjacent to claude.ai Projects)

Re-check official help when UI labels move; product surfaces change faster than blog posts.

Core questions (self-check)

1. **What is an entry point?** — The container that sets context persistence and output form.
2. **What three things does a Project hold?** — Instructions, knowledge base, its own chats.
3. **Do Project chats share context with each other?** — No, unless facts live in knowledge/instructions (or product memory scoped that way).
4. **When is a Project worth building?** — ≥2 of: recur / stable background / consistent format.
5. **Chat vs Research vs web search?** — Chat default; web search for quick live facts; Research for deep multi-source synthesis.
6. **When Artifact?** — When the output is a deliverable to open/edit, not only a conversational reply.
7. **How do you fine-tune a Project?** — Change one instruction or knowledge file after a real failure; retest in a new chat.

Source note

Teaching spine from **Claude Associate · Module 1 · Core Entry Points** (including the weekly status worked comparison). Product limits and setup steps cross-checked against Anthropic Help Center (Projects available on free with a 5-project cap; RAG expansion on paid). Examples B–G and the fine-tune loop are operator expansions for learning/productivity, not course screenshots.